

《Arena Survivor》C++ Qt 大作业项目技术报告

基本信息

- 项目名称：《Arena Survivor》（无尽生存竞技场）
- 项目组号：第 [121] 组
- 团队名称：[Tequila Sunrise]
- 指导教师：[杨帅老师]
- 小组成员与学号：
 - 组长：[张钟渠]（学号：[2500013089]）—— 负责主框架、UI渲染与持久化
 - 组员一：[陈若宇]（学号：[2500013115]）—— 负责核心战斗、武器与几何碰撞
 - 组员二：[杨皓轶]（学号：[2500013051]）—— 负责关卡波次、AI与肉鸽系统

一、项目主要功能概述

《Arena Survivor》是一款基于 **C++17** 与 **Qt Graphics View Framework** 深度构建的俯视角动作割草幸存者射击游戏。本项目围绕“核心战斗—局内养成—循环波次”的经典框架展开，包含了完备且健壮的游戏系统功能：

1. 全周期有限状态机（State Machine）

游戏主控层提供了清晰的状态流转机制，支持以下六大独立状态的无缝切换与互斥排流：

- 主菜单（MainMenu）**：支持难度选择、键盘控制模式切换（方向键/WASD）、BGM切换。
- 游戏进行中（Playing）**：推进实体物理计算、伤害计算与画面视角跟随。
- 游戏暂停（Paused）**：冻结游戏世界时钟，保留视口渲染并支持随时唤醒。
- 设置菜单（Settings）**：提供音量实时交互调节。
- 升级词条选择（Upgrading）**：每通关指定波次触发“随机肉鸽（Roguelike）三选一”属性提升。
- 游戏结算（GameOver）**：统计全局分数，自动比对写入历史排行榜，并提供快速重开（R键）或重选难度功能。

2. 多元武器与弹道判定系统

- 实弹武器（Bullet）**：具备既定飞行初速度、方向向量与过期销毁判定。
- 高能激光（Laser）**：Hitscan（即时命中）穿透判定，在生成帧对直线射线区域内的所有敌人做瞬发伤害。
- 魔法阵（Magic Circle）**：环绕玩家周身做匀速圆周运动，形成范围持续性伤害（AOE）防守力场。

3. 局内成长与波次调度机制

采用波次生成器（WaveManager），动态计算当前波次的生成密度与敌人属性。怪物死亡后生成经验球（ExperienceOrb），玩家通过位移拾取获取分数。当场上怪物全部肃清且满两波次时，自动切入系统强化选词条界面。

4. 数据持久化与自适应音频引擎

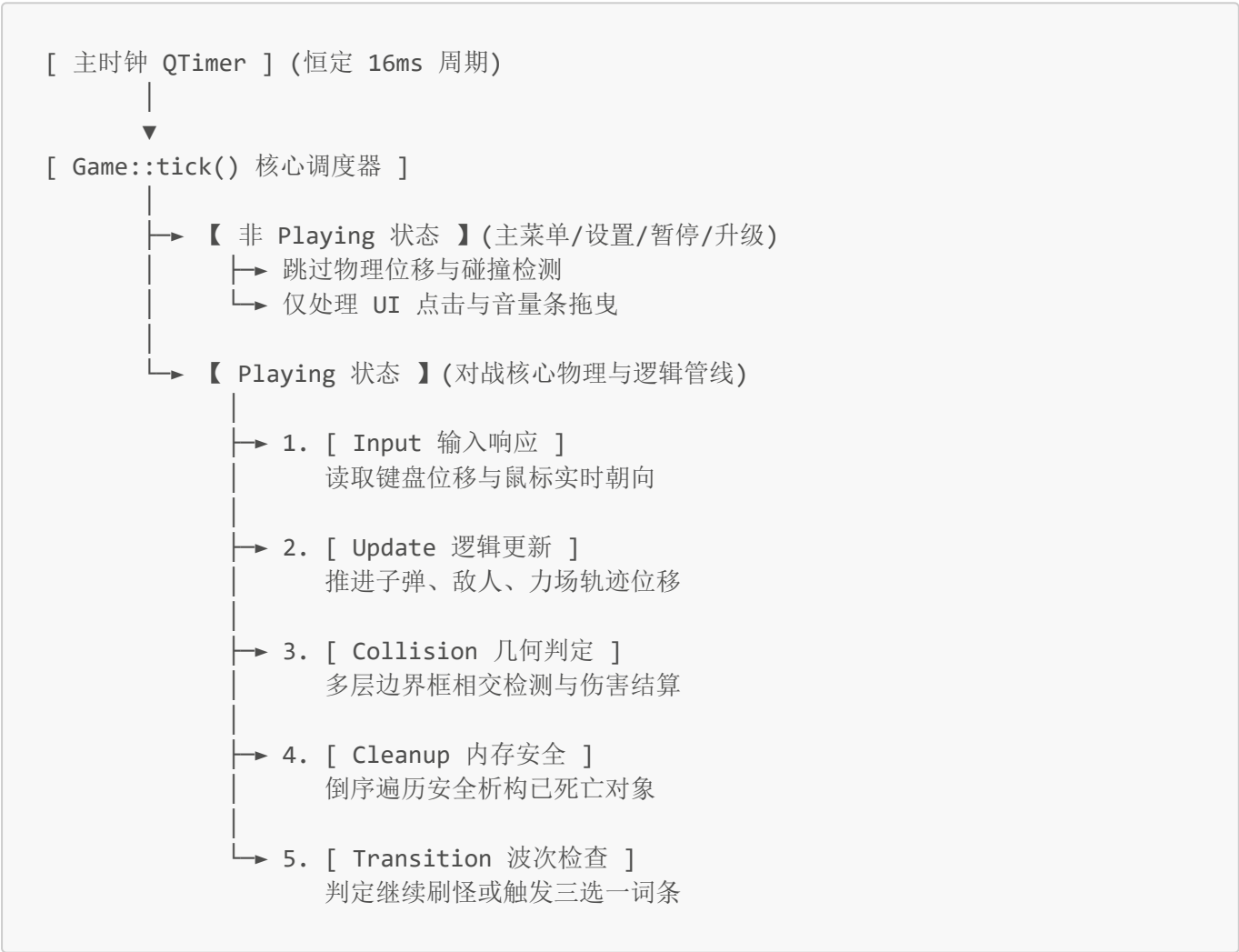
- **历史最高分榜 (High Scores):** 自动根据当前难度读写本地 `highscores.txt` 文件，永久记录前四名好成绩。
- **多级音频目录检索:** 为了防止跨平台或跨 IDE 运行时找不到文件，`QMediaPlayer` 配备了多级相对路径容错检索策略。

二、整体架构设计与核心主循环思路

本项目在设计上严格贯彻了 **MVC (Model-View-Controller)** 架构思想，并将逻辑更新与视图渲染进行了清晰的解耦。整个游戏驱动的核心是基于“固定时间步长 (Fixed Timestep)”的主时钟循环。

1. 游戏主时钟逻辑管线 (Game Loop)

中央控制器 `Game` 实例化时配置了全局定时器 `m_gameTimer`，每隔固定毫秒（如 `TICK_INTERVAL_MS = 16`，约合 60 FPS）恒定触发一次 `Game::tick()` 函数。管线的执行流程如下：



核心调度代码片段如下：

```
void Game::tick()
{
    // 1. 全局热键拦截：无论在哪个状态，按下 R 键均可迅速响应重开游戏
    if (m_view->restartRequested()) {
        m_view->clearRestart();
    }
}
```

```

        if (m_state == GameState::GameOver) {
            restart();
            return;
        }
    }

    // 2. 状态隔离: 如果不是 Playing 状态, 跳过物理和碰撞, 仅处理 UI 渲染与鼠标拖曳
    if (m_state != GameState::Playing) {
        if (m_draggingVolume && m_view->isMouseHeld()) {
            QPointF vp = m_view->mouseScenePos();
            handleVolumeDrag(m_view->mapFromScene(vp));
        }
        m_view->viewport()->update();
        return;
    }

    // 3. 严格的游戏物理与逻辑更新管线
    processInput();           // 接收输入并更新玩家位置、枪口朝向
    updateGameObjects();     // 推进子弹、敌人、经验球及魔法阵位置
    handleCollisions();       // 调用 CollisionManager 执行多层碰撞检测
    cleanupDeadItems();       // 安全清理所有已标记死亡的对象, 释放内存
    checkWaveTransition();    // 检测下一波次开关
    m_view->viewport()->update();
}

```

2. 严密的堆内存管理机制（杜绝野指针与崩溃）

由于游戏中会有大量的子弹倾泻和敌人生成/死亡, 如果直接在循环里正序 `remove` 并 `delete`, 极易导致数组下标错位和迭代器失效问题。

团队在 `cleanupDeadItems()` 模块中统一采用了**自底向上的倒序遍历销毁算法**, 不仅确保容器索引安全, 还在释放敌人内存的同时就地实例化新的经验球对象:

```

void Game::cleanupDeadItems()
{
    // 倒序遍历处理死亡敌军
    for (int i = m_enemies.size() - 1; i >= 0; i--) {
        Enemy *e = m_enemies[i];
        if (e->isDead()) {
            // 在敌军阵亡坐标处生成对应经验值的经验球
            ExperienceOrb *orb = new ExperienceOrb(e->pos(), e->xpValue(),
m_player);
            m_scene->addItem(orb);
            m_orbs.append(orb);

            // 加分并在场景与容器中安全卸载
            m_score += e->scoreValue();
            m_player->addScore(e->scoreValue());
            m_scene->removeItem(e);
            m_enemies.removeAt(i);
            delete e; // 显式释放堆内存, 杜绝泄露
        }
    }
}

```

```
    }  
}  
  
// 子弹、激光与经验球的过期销毁同样遵循此严密标准...  
}
```

三、Qt 核心框架深度对接与技术实践

本项目不仅使用了标准 C++ 面向对象语法，还深度集成了 Qt 框架提供的信号槽机制、视图投影及音频底层接口。

1. 信号与槽（Signals & Slots）实现实体的事件解耦

在复杂的动作游戏中，如果玩家实体（**Player**）直接持有并操作子弹容器或 UI 指针，会导致严重的类依赖耦合。本项目大量利用异步信号机制，实现了“生产实体负责发出通知，场景中央负责接收挂载”的清晰架构：

```
// 在 Game::Game() 构造函数中将玩家动作信号连接至场景调度中心  
connect(m_player, &Player::bulletCreated, this, &Game::onBulletCreated);  
connect(m_player, &Player::laserCreated, this, &Game::onLaserCreated);  
connect(m_player, &Player::playerDied, this, &Game::onPlayerDied);  
  
// 对应回调槽函数具体实现  
void Game::onBulletCreated(Bullet *bullet)  
{  
    m_scene->addItem(bullet); // 将实体挂载至 QGraphicsScene 图形世界  
    m_bullets.append(bullet); // 纳入活跃容器以供物理时钟轮询  
}
```

2. QGraphicsView 视口变换与无缝平滑锁尾

利用 Qt 图形视图框架自带的坐标系转换与摄像机焦点定位机制，轻松实现了自由视角下的玩家镜头追踪：

```
void Game::processInput()  
{  
    m_player->setKeysHeld(m_view->keysPressed());  
    m_player->updateMovement();  
    m_player->updateAiming(m_view->mouseScenePos());  
  
    // 核心调用：实现视口中心始终锚定玩家当前全局物理坐标  
    m_view->centerOn(m_player->pos());  
}
```

3. 自适应多级目录检索的音频驱动引擎

开发和评测过程中，不同的 IDE（如 Qt Creator、VS Code）或构建策略可能会造成应用运行时的当前工作目录（Working Directory）不一致，常引发音频加载失败。为此，在背景音乐模块编写了多级容错检索代码：

```
// 依次向上逐级检索相对目录，提高项目跨电脑运行的鲁棒性
QDir musicDir(QCoreApplication::applicationDirPath() + "/Musics");
if (!musicDir.exists())
    musicDir = QDir(QCoreApplication::applicationDirPath() + "../Musics");
if (!musicDir.exists())
    musicDir = QDir(QCoreApplication::applicationDirPath() + "../../Musics");
if (!musicDir.exists())
    musicDir = QDir("Musics"); // 最终保底方案

if (musicDir.exists()) {
    QStringList filters;
    filters << "*.mp3" << "*.wav" << "*.ogg" << "*.flac";
    m_musicFiles = musicDir.entryList(filters, QDir::Files);
    for (QString &f : m_musicFiles)
        f = musicDir.absoluteFilePath(f); // 转化为操作系统绝对路径
}
```

四、核心算法与创新点解析

1. 瞬发激光（Hitscan）即时命中检测算法

游戏中普通子弹通过逐帧步长位移进行碰撞，而高能激光武器（**Laser**）为了还原“光速即时击中”的爽快感，采用了**Hitscan（即时相交判定）**技术。当激光生成的瞬间，立刻对其直线方向上的所有敌人做相交判定并扣除生命值，后续帧仅作画质残影展示：

```
void Game::onLaserCreated(Laser *laser)
{
    m_scene->addItem(laser);
    // 即时判定：不需等待主物理时钟轮询，立刻计算并结算整条射击线上的相交伤害
    laser->checkHit(m_enemies);
    m_lasers.append(laser);
}
```

2. UI 交互式滑动条的归一化几何插值算法

游戏中摒弃了原生的传统控件堆叠，在底层场景画布中手绘了设置界面的音量交互滑动条。当用户使用鼠标点击或拖拽时，通过获取坐标横向线段差值的归一化比值，实现了对系统音量的线性插值控制：

$$\text{Volume} = \text{clamp} \left(\frac{\text{pos}_x - \text{bar}_{\text{left}}}{\text{bar}_{\text{width}}}, 0.0, 1.0 \right)$$

实现代码片段如下：

```
void Game::handleVolumeDrag(QPointF scenePos)
{
    QRectF bar;
    if (m_state == GameState::MainMenu)
```

```
        bar = HUD::volumeBarRectMainMenu(m_view->viewport()->rect());
    else
        bar = HUD::volumeBarRect(m_view->viewport()->rect());

    // 计算当前鼠标横向位置相对于进度条总宽度的百分比
    double pct = (scenePos.x() - bar.left()) / bar.width();
    pct = qBound(0.0, pct, 1.0); // 严格夹击限定在 0% 到 100% 之间

    m_settings.musicVolume = static_cast<float>(pct);
    m_bgmOutput->setVolume(pct);
    syncHudSettings(); // 触发界面重绘渲染进度条最新插值宽度
}
```

五、项目工作量拆分与三人分工建议

为确保课程设计考核时团队成员能够**平等且充分**地展示各自在 C++ 面向对象编程、几何算法和图形界面等领域的贡献，按照以下矩阵进行了合理分工：

团队角色	负责的核心源码文件	模块职责与具体贡献描述
组长 (基础架构与界面 UI)	Game.cpp / .h GameView.cpp / .h HUD.cpp / .h Constants.h	• 系统框架搭建：负责游戏主控制器、QTimer 物理主循环架构设计与多状态机状态流转控制。 • 内存安全管理：编写 cleanupDeadItems() 中的倒序堆内存回收算法。 • UI/HUD 引擎：自绘主菜单、暂停栏、血量栏以及音量鼠标拖拽归一化运算。 • 持久化模块：负责本地 High Score 排行榜文本文件 I/O 读写。
组员一 (核心战斗与几何碰撞)	Player.cpp / .h Bullet.cpp / .h Laser.cpp / .h MagicCircle.cpp / .h CollisionManager.cpp / .h	• 玩家操作控制：实现键盘位移、八向朝向向量运算以及鼠标实时瞄准反馈逻辑。 • 武器系统：独立开发普通飞行子弹、即时命中激光（Hitscan）与环绕魔法阵三套武器类。 • 碰撞检测核心：主导开发 CollisionManager，编写多边形与圆形的几何相交检测算法。 • 玩家属性维护：编写无敌帧定时闪烁计算及受击扣血结算。
组员二 (AI 关卡与肉鸽系统)	WaveManager.cpp / .h Enemy.cpp / .h ExperienceOrb.cpp / .h UpgradeSystem.cpp / .h TextureManager.cpp / .h	• 敌军 AI 行为：编写怪物追踪索敌逻辑、自动寻路朝向计算与死亡掉落判定。 • 关卡调度：开发 WaveManager 波次管理器，负责动态调控四级难度的刷怪间隔与怪物强度。 • 成长词条系统：实现 UpgradeSystem，负责击杀经验计算与“随机三选一”局内养成逻辑。 • 资源与多媒体：封装多级兼容的目录检索算法，完成音频模块接入与贴图资源初始化。